

Homework 4

Ravi Kini

December 12, 2025

Exercise 1

Part (

a) Let L_n be the degree n Legendre polynomial and $p(x)$ be some polynomial with degree less than n . We can represent $p(x) = \sum_{k=0}^{n-1} a_k x^k$. Then:

$$\begin{aligned} \int_{-1}^1 L_n(x) p(x) dx &= \int_{-1}^1 L_n(x) \left(\sum_{k=0}^{n-1} a_k x^k \right) dx \\ &= \sum_{k=0}^{n-1} a_k \int_{-1}^1 L_n(x) x^k dx \end{aligned}$$

To show that $\int_{-1}^1 L_n(x) p(x) dx = 0$, it is then sufficient to show that $\int_{-1}^1 L_n(x) x^k dx = 0$ for all $k < n$. Using Rodrigue's formula:

$$\begin{aligned} \int_{-1}^1 L_n(x) x^k dx &= \int_{-1}^1 \left(\frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n \right) x^k dx \\ &= \frac{1}{2^n n!} \int_{-1}^1 \left(\frac{d^n}{dx^n} (x^2 - 1)^n \right) x^k dx \end{aligned}$$

Applying integration by parts once:

$$\begin{aligned} \int_{-1}^1 \left(\frac{d^n}{dx^n} (x^2 - 1)^n \right) x^k dx &= \left(\frac{d^{n-1}}{dx^{n-1}} (x^2 - 1)^n \right) x^k dx \Big|_{-1}^1 - \int_{-1}^1 \left(\frac{d^{n-1}}{dx^{n-1}} (x^2 - 1)^n \right) k x^{k-1} dx \\ &= - \int_{-1}^1 \left(\frac{d^{n-1}}{dx^{n-1}} (x^2 - 1)^n \right) k x^{k-1} dx \end{aligned}$$

Clearly by applying integration by parts n times, we get:

$$\int_{-1}^1 \left(\frac{d^n}{dx^n} (x^2 - 1)^n \right) x^k dx = \frac{(-1)^n k!}{\int_{-1}^1} (x^2 - 1)^n \left(\frac{d^n}{dx^n} x^k \right) dx$$

Since $k < n$, $\frac{d^n}{dx^n} x^k = 0$. It then follows that $\int_{-1}^1 L_n(x) x^k dx = 0$ for all $k < n$, and so $\int_{-1}^1 L_n(x) p(x) dx$ for an arbitrary $p(x)$ with degree $k < n$.

Part (b)

Let $\tilde{L}_n$ be the monic degree n Legendre polynomial and $\tilde{p}(x)$ be some monic polynomial with degree n . Their difference $r(x) = \tilde{p}(x) - \tilde{L}_n(x)$ then has degree $k < n$. Since Legendre polynomials are orthogonal to polynomials of smaller degree:

$$\langle \tilde{L}_n, r \rangle = \int_{-1}^1 \tilde{L}_n(x)r(x) dx = 0$$

Now, using the properties of the norm:

$$\begin{aligned} ||\tilde{p}||_2^2 &= \langle \tilde{p}, \tilde{p} \rangle \\ &= \langle \tilde{L}_n + r, \tilde{L}_n + r \rangle \\ &= \langle \tilde{L}_n, \tilde{L}_n \rangle + 2\langle \tilde{L}_n, r \rangle + \langle r, r \rangle \\ &= \langle \tilde{L}_n, \tilde{L}_n \rangle + \langle r, r \rangle \\ &= ||\tilde{L}_n||_2^2 + ||r||_2^2 \end{aligned}$$

Since $||r||_2^2 \geq 0$, $||\tilde{p}||_2^2 \geq ||\tilde{L}_n||_2^2$ for all such $\tilde{p}$.

Exercise 2

Part (a)

Let $p(x) = 2^{-n}(T_{n+1}(x) - T_{n-1}(x))$, where $T_n(x)$ is the n th degree Chebyshev polynomial. Then, letting $x_k = \cos(k\pi/n)$ for $k \in \{0, \dots, n\}$:

$$\begin{aligned} p(x_k) &= 2^{-n}(T_{n+1}(x_k) - T_{n-1}(x_k)) \\ &= 2^{-n}(T_{n+1}(\cos(k\pi/n)) - T_{n-1}(\cos(k\pi/n))) \\ &= 2^{-n}(\cos((n+1)k\pi/n) - \cos((n-1)k\pi/n)) \\ &= 2^{-n}(\cos(k\pi + k\pi/n) - \cos(k\pi - k\pi/n)) \end{aligned}$$

If k is even:

$$\begin{aligned} p(x_k) &= 2^{-n}(\cos(k\pi/n) - \cos(-k\pi/n)) \\ &= 2^{-n}(\cos(k\pi/n) - \cos(k\pi/n)) = 0 \end{aligned}$$

If k is odd:

$$\begin{aligned} p(x_k) &= 2^{-n}(\cos(\pi + k\pi/n) - \cos(\pi - k\pi/n)) \\ &= 2^{-n}(-\cos(k\pi/n) + \cos(-k\pi/n)) \\ &= 2^{-n}(-\cos(k\pi/n) + \cos(k\pi/n)) = 0 \end{aligned}$$

In either case, x_k is a root of $p(x)$ for all k . Note that $p(x)$ has degree $n+1$ (since T_{n+1} is of degree $n+1$, and T_{n-1} is of lesser degree), and so can have at most $n+1$ roots, which must then be $x_0, \dots, x_n$. For the same reason, $T_{n+1}(x) - T_{n-1}(x)$ has leading coefficient 2^n , and so $p(x)$ has leading coefficient $2^{-n} \cdot 2^n = 1$, making it monic. $p(x)$ is then the unique monic polynomial with degree $n+1$ and roots $x_0, \dots, x_n$.

Part (b)

```
1 import numpy as np, scipy as sp
2 import matplotlib.pyplot as plt
3
4 def equally_spaced(n):
5     return np.array([-1 + 2 * k / n for k in range(n + 1)])
6
7 def extrema(n):
8     return np.array([np.cos(k * np.pi / n) for k in range(n + 1)])
9
10 def zeros(n):
11     return np.array([np.cos((2 * k - 1) * np.pi / (2 * (n + 1))) for k in range(1, n + 2)])
12
13 def interpolate(f, n, typ):
14     if typ == 'equally spaced':
15         x = equally_spaced(n)
16     if typ == 'extrema':
17         x = extrema(n)
18     if typ == 'zeros':
```

```

19     x = zeros(n)
20     y = f(x)
21     return sp.interpolate.lagrange(x, y)

```

The following table shows the maximum approximation error when using n equally spaced points to interpolate the functions $f_1(x) = \exp(-(x - 0.5)^2)$, $f_2(x) = (16x^2 + 1)^{-1}$, and $f_3(x) = x|x|$ on the interval $[-1, 1]$:

n	f_1	f_2	f_3
4	$1.911 \cdot 10^{-2}$	$3.853 \cdot 10^{-1}$	$3.208 \cdot 10^{-2}$
8	$2.224 \cdot 10^{-4}$	$7.319 \cdot 10^{-1}$	$1.439 \cdot 10^{-2}$
12	$1.099 \cdot 10^{-6}$	$1.969 \cdot 10^0$	$2.867 \cdot 10^{-2}$
16	$6.835 \cdot 10^{-9}$	$5.911 \cdot 10^0$	$1.071 \cdot 10^{-1}$
20	$12.382 \cdot 10^{-6}$	$1.875 \cdot 10^1$	$5.630 \cdot 10^{-1}$

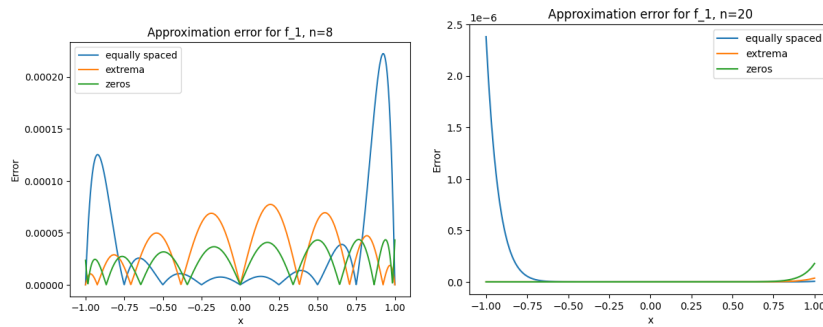
Instead using the extrema of T_n :

n	f_1	f_2	f_3
4	$2.067 \cdot 10^{-2}$	$3.708 \cdot 10^{-1}$	$5.047 \cdot 10^{-2}$
8	$7.735 \cdot 10^{-5}$	$1.307 \cdot 10^{-1}$	$1.198 \cdot 10^{-2}$
12	$9.586 \cdot 10^{-8}$	$4.251 \cdot 10^{-2}$	$5.271 \cdot 10^{-3}$
16	$2.068 \cdot 10^{-10}$	$1.754 \cdot 10^{-2}$	$2.955 \cdot 10^{-3}$
20	$3.614 \cdot 10^{-8}$	$6.671 \cdot 10^{-3}$	$1.888 \cdot 10^{-3}$

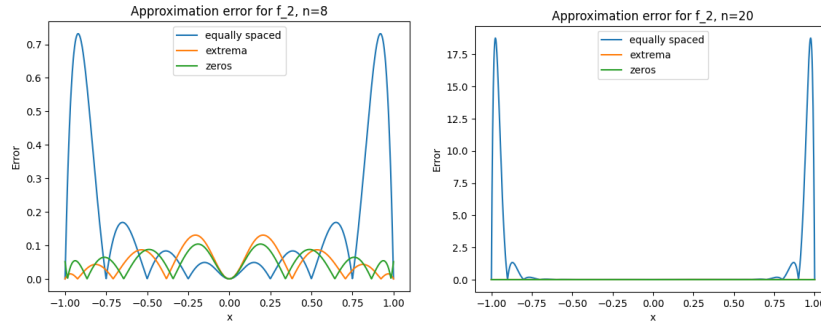
Instead using the positive zeros of T_{n+1} :

n	f_1	f_2	f_3
4	$1.199 \cdot 10^{-2}$	$3.118 \cdot 10^{-1}$	$3.858 \cdot 10^{-2}$
8	$4.361 \cdot 10^{-5}$	$1.039 \cdot 10^{-1}$	$9.995 \cdot 10^{-3}$
12	$6.225 \cdot 10^{-8}$	$3.789 \cdot 10^{-2}$	$4.611 \cdot 10^{-3}$
16	$1.068 \cdot 10^{-9}$	$1.484 \cdot 10^{-2}$	$2.658 \cdot 10^{-3}$
20	$1.785 \cdot 10^{-7}$	$5.498 \cdot 10^{-3}$	$1.730 \cdot 10^{-3}$

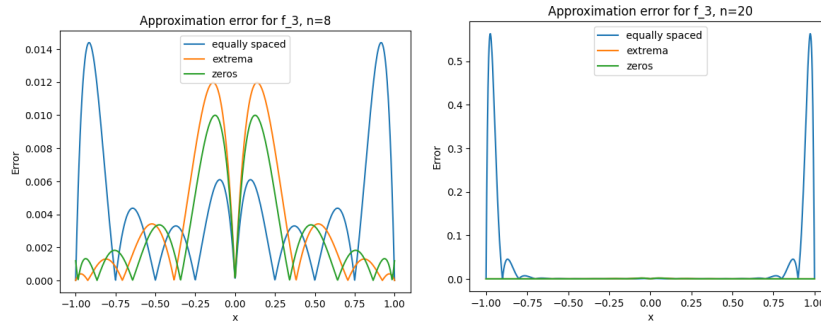
Plotting the approximation error for f_1 , with $n = 8$ and $n = 20$:



For f_2 :



For f_3 :



For f_1 and f_3 , all three methods perform comparably, likely because $f'_1(x) = -2(x - 0.5)\exp(-(x - 0.5)^2)$ and $f'_3(x) = 2x\text{sign}(x)$ do not change very rapidly. For f_2 , using qually spaced points has a much higher maximum approximation error than the other two methods, likely, because $f'_2(x) = -32x(16x^2 + 1)^{-2}$ changes much more rapidly, particularly as it crosses $x = 0$.

Part (c)

Comparing the two types of Chebyshev points used, they produce comparable maximum approximation errors. From the result derived in part (a), since $p(x)$ is the unique monic polynomial with roots at x_k , which are also the extrema of T_n , using those as the interpolation points minimizes the maximum approximation error.

Exercise 3

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def composite_trapezoidal(f, a, b, n):
5     h = (b - a) / n
6     x = np.linspace(a, b, n + 1)
7     y = f(x)
8     return h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
9
10 def composite_simpsons(f, a, b, n):
11     h = (b - a) / n
12     x = np.linspace(a, b, n + 1)
13     y = f(x)
14     return h / 3 * (y[0] + 4 * np.sum(y[1:-1:2]) + 2 * np.sum(y[2:-1:2]) + y[-1])
15
16 def composite_3pt_gaussian_quad(f, a, b, n):
17     n = np.array([-np.sqrt(3/5), 0, np.sqrt(3/5)])
18     w = np.array([5/9, 8/9, 5/9])
19     h = (b - a) / n
20     integral = 0.0
21     for i in range(n):
22         xl, xr = a + i * h, a + (i + 1) * h
23         xm, xh = (xl + xr) / 2, (xr - xl) / 2
24         y = f(xm + xh * n)
25         integral += xh * np.sum(w * y)
26     return integral
27
28 def composite_4pt_gaussian_quad(f, a, b, n):
29     n = np.array([-np.sqrt((3 + 2*np.sqrt(6/5)) / 7), -np.sqrt((3 - 2*np.sqrt(6/5)) / 7), np.sqrt((3 - 2*np.sqrt(6/5)) / 7), np.sqrt((3 + 2*np.sqrt(6/5)) / 7)])
30     w = np.array([(18 - np.sqrt(30)) / 36, (18 + np.sqrt(30)) / 36, (18 + np.sqrt(30)) / 36, (18 - np.sqrt(30)) / 36])
31     h = (b - a) / n
32     integral = 0.0
33     for i in range(n):
34         xl, xr = a + i * h, a + (i + 1) * h
35         xm, xh = (xl + xr) / 2, (xr - xl) / 2
36         y = f(xm + xh * n)
37         integral += xh * np.sum(w * y)
38     return integral

```

The composite trapezoidal rule and composite Simpson's rule require $n + 1$ function evaluations. Composite 3-point Gaussian quadrature and composite 4-point Gaussian quadrature require $3n$ and $4n$ function evaluations, respectively.

Part (b)

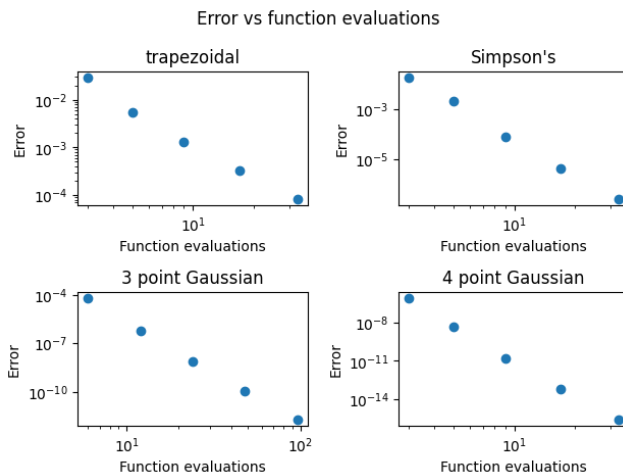
The following table shows the results of approximating $\int_0^1 x \exp(-8x^2) dx$ using the four methods:

n	trapezoidal	Simpson's	3 point	4 point
2	$3.392 \cdot 10^{-2}$	$4.517 \cdot 10^{-2}$	$6.254 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$
4	$5.695 \cdot 10^{-2}$	$6.463 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$
8	$6.115 \cdot 10^{-2}$	$6.256 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$
16	$6.215 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$
32	$6.240 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$	$6.248 \cdot 10^{-2}$

and the following table shows the errors when compared with the actual result of $(1 - e^{-8})/16$:

n	trapezoidal	Simpson's	3 point	4 point
2	$2.856 \cdot 10^{-2}$	$1.731 \cdot 10^{-2}$	$6.042 \cdot 10^{-5}$	$-8.620 \cdot 10^{-7}$
4	$5.529 \cdot 10^{-3}$	$2.148 \cdot 10^{-3}$	$6.251 \cdot 10^{-7}$	$-5.434 \cdot 10^{-9}$
8	$1.325 \cdot 10^{-3}$	$7.619 \cdot 10^{-5}$	$7.886 \cdot 10^{-9}$	$-1.573 \cdot 10^{-11}$
16	$3.282 \cdot 10^{-4}$	$4.157 \cdot 10^{-6}$	$1.177 \cdot 10^{-10}$	$-5.779 \cdot 10^{-14}$
32	$8.185 \cdot 10^{-5}$	$2.521 \cdot 10^{-7}$	$1.819 \cdot 10^{-12}$	$-2.151 \cdot 10^{-16}$

Plotting the error in the approximation against the number of function evaluations for each of the methods:



We see that the composite trapezoidal method converges at a rate of approximately $O(n^{2.4})$, composite Simpson's at a rate of approximately $O(n^{4.7})$, 3 point Gaussian quadrature at a rate of approximately $O(n^{6.2})$, and 4 point Gaussian quadrature at a rate of approximately $O(n^{9.2})$. Evidently, 4 point Gaussian quadrature converges the fastest, and trapezoidal the slowest, with the both requiring a number of function evaluations that is linear in n .

Part (c)

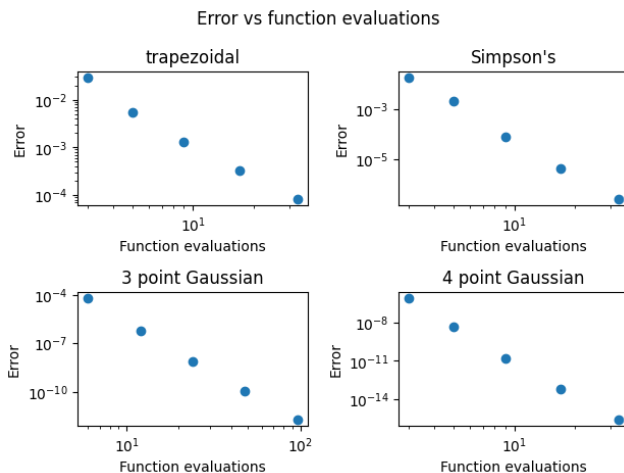
The following table shows the results of approximating $\int_0^1 \sqrt{x} dx$ using the four methods:

n	trapezoidal	Simpson's	3 point	4 point
2	$6.036 \cdot 10^{-1}$	$6.381 \cdot 10^{-1}$	$6.676 \cdot 10^{-1}$	$6.671 \cdot 10^{-1}$
4	$6.433 \cdot 10^{-1}$	$6.565 \cdot 10^{-1}$	$6.670 \cdot 10^{-1}$	$6.668 \cdot 10^{-1}$
8	$6.581 \cdot 10^{-1}$	$6.631 \cdot 10^{-1}$	$6.668 \cdot 10^{-1}$	$6.667 \cdot 10^{-1}$
16	$6.636 \cdot 10^{-1}$	$6.654 \cdot 10^{-1}$	$6.667 \cdot 10^{-1}$	$6.667 \cdot 10^{-1}$
32	$6.656 \cdot 10^{-1}$	$6.662 \cdot 10^{-1}$	$6.667 \cdot 10^{-1}$	$6.667 \cdot 10^{-1}$

and the following table shows the errors when compared with the actual result of $2/3$:

n	trapezoidal	Simpson's	3 point	4 point
2	$6.311 \cdot 10^{-2}$	$2.860 \cdot 10^{-2}$	$8.888 \cdot 10^{-4}$	$4.105 \cdot 10^{-4}$
4	$2.338 \cdot 10^{-2}$	$1.014 \cdot 10^{-2}$	$3.142 \cdot 10^{-4}$	$1.451 \cdot 10^{-4}$
8	$8.536 \cdot 10^{-3}$	$3.587 \cdot 10^{-3}$	$1.111 \cdot 10^{-4}$	$5.131 \cdot 10^{-5}$
16	$3.085 \cdot 10^{-3}$	$1.268 \cdot 10^{-3}$	$3.928 \cdot 10^{-5}$	$1.814 \cdot 10^{-5}$
32	$1.108 \cdot 10^{-3}$	$4.485 \cdot 10^{-4}$	$1.389 \cdot 10^{-5}$	$6.414 \cdot 10^{-6}$

Plotting the error in the approximation against the number of function evaluations for each of the methods:



We see that the composite trapezoidal method converges at a rate of approximately $O(n^{1.7})$, composite Simpson's at a rate of approximately $O(n^{1.7})$, 3 point Gaussian quadrature at a rate of approximately $O(n^{1.5})$, and 4 point Gaussian quadrature at a rate of approximately $O(n^{1.7})$. All four methods converge slower for this problem, notably with 3 point Gaussian quadrature converging the slowest. This is likely because this function is not smooth, as its derivative is singular at $x = 0$.

Exercise 4

Part (a)

```

1 import numpy as np
2
3 func_eval = 0
4
5 def simps(a, b, fa, fm, fb):
6     h = (b - a) / 2
7     return h / 3 * (fa + 4 * fm + fb)
8
9 def adapt(f, a, b, sab, fa, fab2, fb, tol, nodes):
10    global func_eval
11    xab2 = (a + b) / 2
12    xm1, xm2 = (3 * a + b) / 4, (a + 3 * b) / 4
13    fm1, fm2 = f(xm1), f(xm2)
14    func_eval += 2
15    new_nodes = nodes + [xm1, xm2]
16    sm1, sm2 = simps(a, xab2, fa, fm1, fab2), simps(xab2, b, fab2, fm2, fb)
17    factor = 15
18    if abs(sm1 + sm2 - sab) < factor * tol:
19        q = sm1 + sm2
20    else:
21        q1, new_nodes = adapt(f, a, xab2, sm1, fa, fm1, fab2, tol / 2, new_nodes)
22        q2, new_nodes = adapt(f, xab2, b, sm2, fab2, fm2, fb, tol / 2, new_nodes)
23        q = q1 + q2
24    return q, new_nodes
25
26 def adaptive_simpson(f, a, b, tol):
27    global func_eval
28    fa, fab2, fb = f(a), f((a + b) / 2), f(b)
29    func_eval += 3
30    sab = simps(a, b, fa, fab2, fb)
31    nodes = [a, (a + b) / 2, b]
32    q, new_nodes = adapt(f, a, b, sab, fa, fab2, fb, tol, nodes)
33    return q, new_nodes, func_eval

```

The following table shows the errors of approximating $\int_0^1 \frac{1}{x+0.001} dx$ using adaptive Simpson's quadrature when compared with the actual result of $\ln 1001$:

Tolerance	10^{-4}	10^{-6}	10^{-8}	10^{-10}
Error	$2.527 \cdot 10^{-5}$	$3.734 \cdot 10^{-7}$	$3.341 \cdot 10^{-9}$	$3.519 \cdot 10^{-11}$
Function evaluations	161	509	1617	5057

As the tolerance decreases, the number of required function evaluations grows. Checking the set of points at which the function was evaluated, several of the points are close to $x = 0$; this is because the derivative of the integrand is large in this region.

Part (b)

Let $h = 1/n$ and $x_k = (k-1)/h$. We then define $g_k(t)$ such that:

$$g_k(t) = \frac{t}{2}(f(x_k) + f(x_k + t)) - \int_{x_k}^{x_k+t} f(x) dx$$

We see that $g_k(h)$ is the error of the trapezoidal rule on the interval $[x_k, x_k + h]$. Then:

$$\begin{aligned}\frac{dg_k}{dt} &= \frac{1}{2}(f(x_k) + f(x_k + t)) + \frac{t}{2}f'(x_k + t) - f(x_k + t) \\ \frac{d^2g_k}{dt^2} &= \frac{t}{2}f''(x_k + t)\end{aligned}$$

Let $\xi \in [0, 1]$ such that $f''(\xi)$ is maximum. Since the function is sufficiently smooth, $|f''(x)| \leq |f''(\xi)|$ for all $x \in [0, 1]$, and so $|f''(x_k + t)| \leq |f''(\xi)|$. Then, since $g'_k(0), g_k(0) = 0$:

$$\begin{aligned}\int_0^t g''_k(t') dt' &= g'_k(t) \\ \int_0^t g'_k(t') dt' &= g_k(t)\end{aligned}$$

It follows, integrating over t that:

$$\begin{aligned}-\frac{t}{2}f''(\xi) &\leq g''_k(t) \leq \frac{t}{2}f''(\xi) \\ -\frac{t^2}{4}f''(\xi) &\leq g'_k(t) \leq \frac{t^2}{4}f''(\xi) \\ -\frac{t^3}{12}f''(\xi) &\leq g_k(t) \leq \frac{t^3}{12}f''(\xi) \\ -\frac{h^3}{12}f''(\xi) &\leq g_k(h) \leq \frac{h^3}{12}f''(\xi)\end{aligned}$$

Summing over the error terms for all of the intervals:

$$\begin{aligned}-\sum_{k=1}^n \frac{h^3}{12}f''(\xi) &\leq \sum_{k=1}^n g_k(h) \leq \sum_{k=1}^n \frac{h^3}{12}f''(\xi) \\ -n\frac{h^3}{12}f''(\xi) &\leq \sum_{k=1}^n g_k(h) \leq n\frac{h^3}{12}f''(\xi) \\ \left| \sum_{k=1}^n g_k(h) \right| &\leq \frac{1}{12n^2}|f''(\xi)|\end{aligned}$$

If $f(x) = \frac{1}{x+0.001}$, $f''(x) = \frac{2}{(x+0.001)^3}$, which is maximum at $\xi = 0$, where $f''(\xi) = 2 \cdot 10^9$. We then see that the error is bounded by:

$$\text{err} \leq \frac{1}{6n^2} \cdot 10^9$$

To guarantee that the error is below 10^{-10} using the composite trapezoidal rule, it follows that the number of points required is upper bounded by $\sqrt{10^{19}/6} \approx 1.29 \cdot 10^9$.

We now instead define $g_k(t)$ such that:

$$g_k(t) = \frac{t}{3}(f(x_k - t) + 4f(x_k) + f(x_k + t)) - \int_{x_k - t}^{x_k + t} f(x) dx$$

We see that $g_k(h/2)$ is the error of Simpson's rule on the interval $[x_k - h/2, x_k + h/2]$. Then:

$$\begin{aligned} \frac{dg_k}{dt} &= \frac{1}{3}(-2f(x_k - t) + 4f(x_k) - 2f(x_k + t)) - \frac{t}{3}(f'(x_k - t) - f'(x_k + t)) \\ \frac{d^2g_k}{dt^2} &= \frac{1}{3}(f'(x_k - t) - f'(x_k + t)) + \frac{t}{3}(f''(x_k - t) + f''(x_k + t)) \\ \frac{d^3g_k}{dt^3} &= -\frac{t}{3}(f^{(3)}(x_k + t) - f^{(3)}(x_k - t)) \end{aligned}$$

Let $\xi \in [0, 1]$ such that $f^{(4)}(\xi)$ is maximum. Since the function is sufficiently smooth, $|f^{(4)}(x)| \leq |f^{(4)}(\xi)|$ for all $x \in [0, 1]$, and so $|f^{(4)}(x_k + t)| \leq |f^{(4)}(\xi)|$. Then, since $g_k''(0), g_k'(0), g_k(0) = 0$, applying the extended mean theorem, for some t', t'' :

$$\begin{aligned} \frac{g_k(t)}{t^5} &= \frac{g^{(3)}(t')}{60t'^2} \\ &= -\frac{1}{90t'}(f^{(3)}(x_k + t') - f^{(3)}(x_k - t')) \\ &= -\frac{1}{90}f^{(4)}(t'') \end{aligned}$$

Then:

$$\begin{aligned} |g_k(t)| &\leq \left| \frac{t^5}{90} f^{(4)}(\xi) \right| \\ |g_k(h)| &\leq \left| \frac{h^5}{90} f^{(4)}(\xi) \right| \end{aligned}$$

Summing over the error terms for all of the intervals:

$$\begin{aligned} \left| \sum_{k=1}^n g_k(h) \right| &\leq \sum_{k=1}^n |g_k(h)| \leq \sum_{k=1}^n \left| \frac{h^5}{90} f^{(4)}(\xi) \right| \\ &\leq n \left| \frac{h^5}{90} f^{(4)}(\xi) \right| \\ &\leq \frac{1}{90n^5} |f^{(4)}(\xi)| \end{aligned}$$

If $f(x) = \frac{1}{x+0.001}$, $f^{(4)}(x) = \frac{24}{(x+0.001)^5}$, which is maximum at $\xi = 0$, where $f''(\xi) = 2 \cdot 10^{15}$. We then see that the error is bounded by:

$$\text{err} \leq \frac{4}{15n^5} \cdot 10^{15}$$

To guarantee that the error is below 10^{-10} using the composite Simpson's rule, it follows that the number of points required is upper bounded by $\sqrt[5]{4 \cdot 10^{25}/15} \approx 76770$.

Part (c)

```
1 err = 100
2 n = 1
3 while err > 1e-10:
4     res = composite_simpsons(f, 0, 1, n)
5     err = abs(res - np.log(1001))
6     n += 1
7 print(res, err, n)
```

In reality, only 5057 points are required to get the error below 10^{-10} . This is much lower than the analytic bound, which is likely a product of the high values of the derivatives around $\xi = 0$ to get a precise approximation close to this point.